PROJECT BASED EXPERIMENTAL LEARNING

COURSE- DATABASE MANAGEMENT SYSTEM (NoSQL / MongoDB)

PROJECT REPORT

TOPIC- MONGODB STUDENT DATABASE OPERATIONS



SUBMITTED TO- MR. Harendra Singh Rajpoot

Submitted by- Yash Tandon

BCADS27

Roll no. - 1250258513

3rd semester

Babu Banarasi Das University

Lucknow, U.P

ACKNOWLEDGMENT

I would like to express my special thanks of gratitude to my mentor Mr. Harendra Singh Rajpoot, who gave me the golden opportunity to do this wonderful project on MongoDB Student Database Operations. My mentor helped me a lot in completing this project within the given time frame, and I learned many new things about NoSQL databases from him. He converted his experience into our knowledge.

Secondly, I want to show my gratitude to IBM, which provided me this project-based learning experience; it helped me to achieve my goals more efficiently.

Abstract

This report presents a MongoDB-based Student Database Management project that demonstrates the core Create, Read, Update and Delete (CRUD) operations of a NoSQL document database. A "students" collection is used to store student records containing roll number, name, age, department and marks. The project implements single and bulk insertion, conditional updates, single and bulk deletion, and a wide range of query operations using MongoDB comparison and logical operators such as $gt, $lt, $eq, $gte, $ne, $and, $or and $not. Every operation is documented with the exact command and a screenshot of the output produced in the MongoDB shell (mongosh). The implementation demonstrates how MongoDB can be used to efficiently manage, filter and retrieve structured student data.

Introduction

MongoDB is a popular NoSQL, document-oriented database that stores data as flexible, JSON-like documents rather than in rigid rows and columns. This makes it well suited for applications where the data structure may evolve over time, such as academic record-keeping systems. This project focuses on building and querying a "students" collection that stores academic details of 50 students across the DS, IoT, Cyber and Cloud departments (14 DS, 13 IoT, 12 Cyber and 11 Cloud at the time of insertion), and demonstrates the standard database operations - insertion, updation, deletion and querying - that form the backbone of any real-world database-driven application.

Methodology

The project was carried out step by step inside the MongoDB shell (mongosh), with every operation executed against a single collection named "students" in a database created for this purpose. The complete methodology, along with the exact code executed and a screenshot of the output produced by MongoDB at each step, is given below.

1. Insert One

Inserts a single student document (roll number 301) into the students collection.

Code:

```
db.students.insertOne({
  rollNo: 301,
  name: "Aarohi",
  age: 20,
  department: "DS",
  marks: 72
})
```

Output:

```
> db.students.insertOne({ rollNo: 301, name: "Aarohi", age: 20, department: "DS", marks: 72 })
{
  acknowledged: true,
  insertedId: ObjectId('6acf2ff1dd73cf256d8f4d3e')
}
```

Figure 1: Output of insertOne()

2. Insert Many

Inserts multiple student documents (roll numbers 302 to 350) into the collection at the same time.

Code:

```
db.students.insertMany([
  { rollNo: 302, name: "Vihaan", age: 19, department: "IoT", marks: 69 },
  { rollNo: 303, name: "Siddharth", age: 22, department: "Cyber", marks: 91 },
  { rollNo: 304, name: "Tanvi", age: 22, department: "Cloud", marks: 92 },
  { rollNo: 305, name: "Kabir", age: 22, department: "DS", marks: 70 },
  { rollNo: 306, name: "Anushka", age: 19, department: "IoT", marks: 94 },
  { rollNo: 307, name: "Reyansh", age: 19, department: "Cyber", marks: 62 },
  { rollNo: 308, name: "Mahi", age: 19, department: "Cyber", marks: 83 },
  { rollNo: 309, name: "Ayaan", age: 19, department: "DS", marks: 98 },
  { rollNo: 310, name: "Prisha", age: 22, department: "IoT", marks: 85 },
  { rollNo: 311, name: "Dhruv", age: 21, department: "Cyber", marks: 68 },
  { rollNo: 312, name: "Navya", age: 22, department: "Cloud", marks: 71 },
  { rollNo: 313, name: "Kian", age: 21, department: "DS", marks: 77 },
  { rollNo: 314, name: "Saanvi", age: 22, department: "IoT", marks: 87 },
  { rollNo: 315, name: "Arjun", age: 21, department: "DS", marks: 60 },
  { rollNo: 316, name: "Riddhi", age: 21, department: "Cloud", marks: 73 },
  { rollNo: 317, name: "Veer", age: 22, department: "DS", marks: 35 },
  { rollNo: 318, name: "Aditi", age: 21, department: "IoT", marks: 56 },
  { rollNo: 319, name: "Yuvraj", age: 21, department: "Cyber", marks: 78 },
  { rollNo: 320, name: "Kiara", age: 22, department: "Cloud", marks: 82 },
  { rollNo: 321, name: "Lakshya", age: 19, department: "DS", marks: 80 },
  { rollNo: 322, name: "Pari", age: 19, department: "Cloud", marks: 82 },
  { rollNo: 323, name: "Rudra", age: 22, department: "Cyber", marks: 80 },
  { rollNo: 324, name: "Ananya", age: 21, department: "Cloud", marks: 81 },
  { rollNo: 325, name: "Shaurya", age: 22, department: "DS", marks: 87 },
  { rollNo: 326, name: "Ishani", age: 19, department: "IoT", marks: 91 },
  { rollNo: 327, name: "Atharv", age: 20, department: "Cyber", marks: 63 },
  { rollNo: 328, name: "Myra", age: 19, department: "Cloud", marks: 88 },
  { rollNo: 329, name: "Daksh", age: 22, department: "DS", marks: 85 },
  { rollNo: 330, name: "Sneha", age: 20, department: "IoT", marks: 84 },
  { rollNo: 331, name: "Parth", age: 20, department: "Cyber", marks: 62 },
  { rollNo: 332, name: "Trisha", age: 22, department: "Cloud", marks: 98 },
  { rollNo: 333, name: "Krish", age: 22, department: "DS", marks: 89 },
  { rollNo: 334, name: "Vedika", age: 21, department: "IoT", marks: 90 },
  { rollNo: 335, name: "Neel", age: 22, department: "Cyber", marks: 97 },
  { rollNo: 336, name: "Jiya", age: 22, department: "Cyber", marks: 80 },
  { rollNo: 337, name: "Samar", age: 22, department: "DS", marks: 67 },
  { rollNo: 338, name: "Khushi", age: 21, department: "IoT", marks: 38 },
  { rollNo: 339, name: "Om", age: 19, department: "Cyber", marks: 57 },
  { rollNo: 340, name: "Aarna", age: 20, department: "Cloud", marks: 96 },
  { rollNo: 341, name: "Devansh", age: 22, department: "DS", marks: 88 },
  { rollNo: 342, name: "Mira", age: 22, department: "IoT", marks: 78 },
  { rollNo: 343, name: "Hriday", age: 21, department: "IoT", marks: 99 },
  { rollNo: 344, name: "Tara", age: 22, department: "Cloud", marks: 84 },
  { rollNo: 345, name: "Ronit", age: 22, department: "DS", marks: 82 },
  { rollNo: 346, name: "Zainab", age: 19, department: "IoT", marks: 73 },
  { rollNo: 347, name: "Ayush", age: 21, department: "Cyber", marks: 84 },
  { rollNo: 348, name: "Fatima", age: 19, department: "Cloud", marks: 83 },
  { rollNo: 349, name: "Nitin", age: 21, department: "DS", marks: 83 },
  { rollNo: 350, name: "Bhavya", age: 22, department: "IoT", marks: 79 }
])
```

Output:

```
> db.students.insertMany([ { rollNo: 302, ... }, ... { rollNo: 350, ... } ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6acf2ff1dd73cf256d8f4d3f'),
    '1': ObjectId('6acf2ff1dd73cf256d8f4d40'),
    '2': ObjectId('6acf2ff1dd73cf256d8f4d41'),
    '3': ObjectId('6acf2ff1dd73cf256d8f4d42'),
    '4': ObjectId('6acf2ff1dd73cf256d8f4d43'),
    // ... 44 more entries ('5' to '48') not shown
  }
}
```

Figure 2: Output of insertMany() - the shell returns one ObjectId per inserted document (list shortened here)

3. Update One

Updates the first document that matches the given condition - here, the marks of roll number 301 are updated from 72 to 78.

Code:

```
db.students.updateOne(
  { rollNo: 301 },
  { $set: { marks: 78 } }
)
```

Output:

```
> db.students.updateOne({ rollNo: 301 }, { $set: { marks: 78 } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Figure 3: Output of updateOne()

4. Update Many

Updates all documents that match the given condition - here, every student in the Cyber department is given a new status field set to "Active".

Code:

```
db.students.updateMany(
  { department: "Cyber" },
  { $set: { status: "Active" } }
)
```

Output:

```
> db.students.updateMany({ department: "Cyber" }, { $set: { status: "Active" } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 12,
  modifiedCount: 12,
  upsertedCount: 0
}
```

Figure 4: Output of updateMany() - all 12 Cyber documents modified

Verification of the updates:

```
db.students.findOne({ rollNo: 301 })
```

Output:

```
> db.students.findOne({ rollNo: 301 })
{
  _id: ObjectId('6acf2ff1dd73cf256d8f4d3e'),
  rollNo: 301,
  name: 'Aarohi',
  age: 20,
  department: 'DS',
  marks: 78
}
```

Figure 5: Roll no. 301 now shows marks: 78

5. Delete One

Deletes the first document that matches the given condition - here, the record of roll number 350 is removed.

Code:

```
db.students.deleteOne({ rollNo: 350 })
```

Output:

```
> db.students.deleteOne({ rollNo: 350 })
{
  acknowledged: true,
  deletedCount: 1
}
```

Figure 6: Output of deleteOne()

6. Delete Many

Deletes all documents that match the given condition - here, every student with marks less than 40 is removed. Two students matched: roll 317 (35 marks) and roll 338 (38 marks).

Code:

```
db.students.deleteMany({ marks: { $lt: 40 } })
```

Output:

```
> db.students.deleteMany({ marks: { $lt: 40 } })
{
  acknowledged: true,
  deletedCount: 2
}
```

Figure 7: Output of deleteMany() - deletedCount is 2

Checking that the deleted records are gone:

```
db.students.find({ rollNo: { $in: [317, 338, 350] } })
```

Output:

```
> db.students.find({ rollNo: { $in: [317, 338, 350] } })
// no documents - all three were deleted
```

Figure 8: No document is returned for the three deleted roll numbers

Total documents after all update and delete operations:

```
db.students.countDocuments()
```

Output:

```
> db.students.countDocuments()
47
```

Figure 9: 47 documents remain (50 inserted - 1 - 2 deleted)

7. Query Operations

A series of find() queries were run using MongoDB's comparison and logical operators to filter and retrieve student records. For each query, the screenshot shows the shell output (the first three matching documents are captured; the shell prints the rest in the same format) and the table lists every matching record.

Marks greater than 80:

Code:

```
db.students.find({ marks: { $gt: 80 } })
```

Output:

```
> db.students.find({ marks: { $gt: 80 } })
{
  _id: ObjectId('6acf2ff1dd73cf256d8f4d40'),
  rollNo: 303,
  name: 'Siddharth',
  age: 22,
  department: 'Cyber',
  marks: 91,
  status: 'Active'
}
{
  _id: ObjectId('6acf2ff1dd73cf256d8f4d41'),
  rollNo: 304,
  name: 'Tanvi',
  age: 22,
  department: 'Cloud',
  marks: 92
}
{
  _id: ObjectId('6acf2ff1dd73cf256d8f4d43'),
  rollNo: 306,
  name: 'Anushka',
  age: 19,
  department: 'IoT',
  marks: 94
}
```

Figure 10: Marks greater than 80 - 27 documents matched (first 3 shown)

All matching records (27):

Roll No	Name	Age	Dept	Marks
303	Siddharth	22	Cyber	91
304	Tanvi	22	Cloud	92
306	Anushka	19	IoT	94
308	Mahi	19	Cyber	83
309	Ayaan	19	DS	98
310	Prisha	22	IoT	85
314	Saanvi	22	IoT	87
320	Kiara	22	Cloud	82
322	Pari	19	Cloud	82
324	Ananya	21	Cloud	81
325	Shaurya	22	DS	87
326	Ishani	19	IoT	91
328	Myra	19	Cloud	88
329	Daksh	22	DS	85

Roll No	Name	Age	Dept	Marks
330	Sneha	20	IoT	84
332	Trisha	22	Cloud	98
333	Krish	22	DS	89
334	Vedika	21	IoT	90
335	Neel	22	Cyber	97
340	Aarna	20	Cloud	96
341	Devansh	22	DS	88
343	Hriday	21	IoT	99
344	Tara	22	Cloud	84
345	Ronit	22	DS	82
347	Ayush	21	Cyber	84
348	Fatima	19	Cloud	83
349	Nitin	21	DS	83

Marks less than 80:

Code:

```
db.students.find({ marks: { $lt: 80 } })
```

Output:

```
> db.students.find({ marks: { $lt: 80 } })
{
  _id: ObjectId('6acf2ff1dd73cf256d8f4d3e'),
  rollNo: 301,
  name: 'Aarohi',
  age: 20,
  department: 'DS',
  marks: 78
}
{
  _id: ObjectId('6acf2ff1dd73cf256d8f4d3f'),
  rollNo: 302,
  name: 'Vihaan',
  age: 19,
  department: 'IoT',
  marks: 69
}
{
  _id: ObjectId('6acf2ff1dd73cf256d8f4d42'),
  rollNo: 305,
  name: 'Kabir',
  age: 22,
  department: 'DS',
  marks: 70
}
```

Figure 11: Marks less than 80 - 17 documents matched (first 3 shown)

All matching records (17):

Roll No	Name	Age	Dept	Marks
301	Aarohi	20	DS	78
302	Vihaan	19	IoT	69
305	Kabir	22	DS	70
307	Reyansh	19	Cyber	62
311	Dhruv	21	Cyber	68
312	Navya	22	Cloud	71
313	Kian	21	DS	77
315	Arjun	21	DS	60
316	Riddhi	21	Cloud	73

Roll No	Name	Age	Dept	Marks
318	Aditi	21	IoT	56
319	Yuvraj	21	Cyber	78
327	Atharv	20	Cyber	63
331	Parth	20	Cyber	62
337	Samar	22	DS	67
339	Om	19	Cyber	57
342	Mira	22	IoT	78
346	Zainab	19	IoT	73

Marks equal to 85:

Code:

```
db.students.find({ marks: { $eq: 85 } })
```

Output:

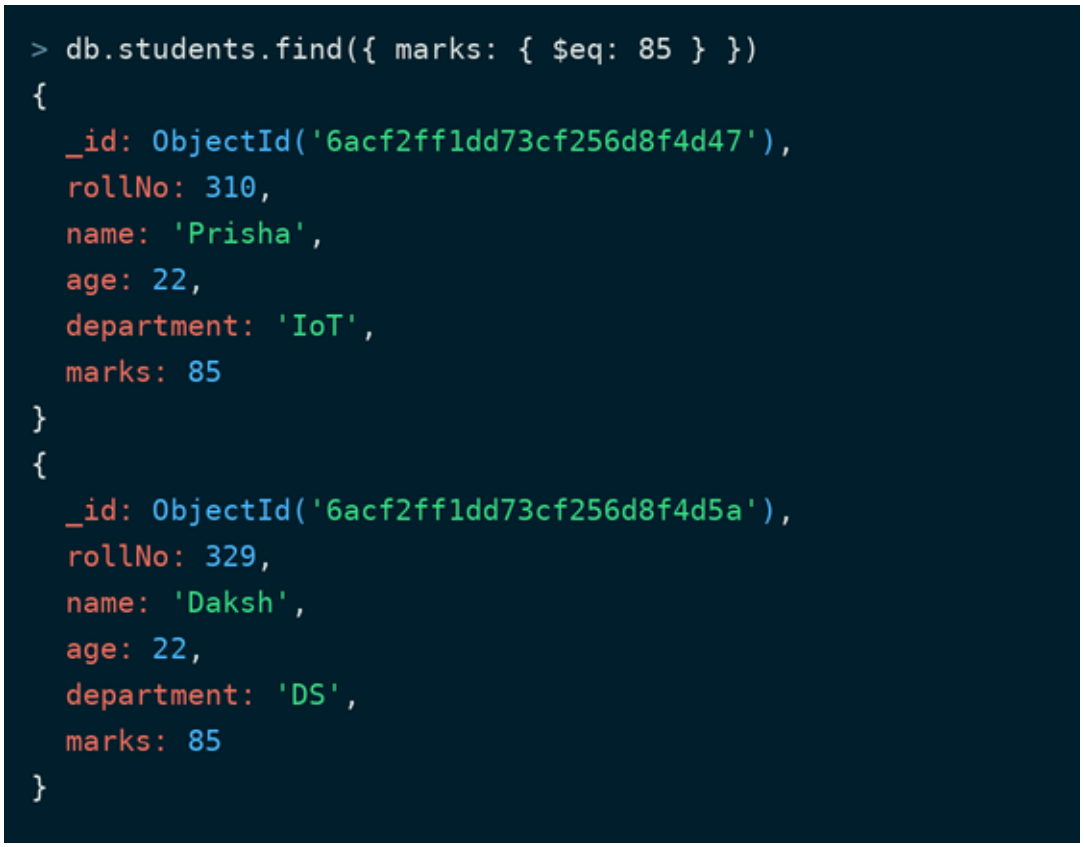


Figure 12: Marks equal to 85 - 2 documents matched (first 3 shown)

All matching records (2):

Roll No	Name	Age	Dept	Marks
310	Prisha	22	IoT	85
329	Daksh	22	DS	85

Marks greater than or equal to 80:

Code:

```
db.students.find({ marks: { $gte: 80 } })
```

Output:

```
> db.students.find({ marks: { $gte: 80 } })
{
  _id: ObjectId('6acf2ff1dd73cf256d8f4d40'),
  rollNo: 303,
  name: 'Siddharth',
  age: 22,
  department: 'Cyber',
  marks: 91,
  status: 'Active'
}
{
  _id: ObjectId('6acf2ff1dd73cf256d8f4d41'),
  rollNo: 304,
  name: 'Tanvi',
  age: 22,
  department: 'Cloud',
  marks: 92
}
{
  _id: ObjectId('6acf2ff1dd73cf256d8f4d43'),
  rollNo: 306,
  name: 'Anushka',
  age: 19,
  department: 'IoT',
  marks: 94
}
```

Figure 13: Marks greater than or equal to 80 - 30 documents matched (first 3 shown)

All matching records (30):

Roll No	Name	Age	Dept	Marks
303	Siddharth	22	Cyber	91
304	Tanvi	22	Cloud	92
306	Anushka	19	IoT	94
308	Mahi	19	Cyber	83
309	Ayaan	19	DS	98
310	Prisha	22	IoT	85
314	Saanvi	22	IoT	87
320	Kiara	22	Cloud	82
321	Lakshya	19	DS	80
322	Pari	19	Cloud	82
323	Rudra	22	Cyber	80
324	Ananya	21	Cloud	81
325	Shaurya	22	DS	87
326	Ishani	19	IoT	91
328	Myra	19	Cloud	88

Roll No	Name	Age	Dept	Marks
329	Daksh	22	DS	85
330	Sneha	20	IoT	84
332	Trisha	22	Cloud	98
333	Krish	22	DS	89
334	Vedika	21	IoT	90
335	Neel	22	Cyber	97
336	Jiya	22	Cyber	80
340	Aarna	20	Cloud	96
341	Devansh	22	DS	88
343	Hriday	21	IoT	99
344	Tara	22	Cloud	84
345	Ronit	22	DS	82
347	Ayush	21	Cyber	84
348	Fatima	19	Cloud	83
349	Nitin	21	DS	83

Marks not equal to 85:

Code:

```
db.students.find({ marks: { $ne: 85 } })
```

Output:

```
> db.students.find({ marks: { $ne: 85 } })
{
  _id: ObjectId('6acf2ff1dd73cf256d8f4d3e'),
  rollNo: 301,
  name: 'Aarohi',
  age: 20,
  department: 'DS',
  marks: 78
}
{
  _id: ObjectId('6acf2ff1dd73cf256d8f4d3f'),
  rollNo: 302,
  name: 'Vihaan',
  age: 19,
  department: 'IoT',
  marks: 69
}
{
  _id: ObjectId('6acf2ff1dd73cf256d8f4d40'),
  rollNo: 303,
  name: 'Siddharth',
  age: 22,
  department: 'Cyber',
  marks: 91,
  status: 'Active'
}
```

Figure 14: Marks not equal to 85 - 45 documents matched (first 3 shown)

All matching records (45):

Roll No	Name	Age	Dept	Marks
301	Aarohi	20	DS	78
302	Vihaan	19	IoT	69
303	Siddharth	22	Cyber	91
304	Tanvi	22	Cloud	92
305	Kabir	22	DS	70
306	Anushka	19	IoT	94
307	Reyansh	19	Cyber	62
308	Mahi	19	Cyber	83
309	Ayaan	19	DS	98
311	Dhruv	21	Cyber	68
312	Navya	22	Cloud	71
313	Kian	21	DS	77
314	Saanvi	22	IoT	87
315	Arjun	21	DS	60
316	Riddhi	21	Cloud	73
318	Aditi	21	IoT	56
319	Yuvraj	21	Cyber	78
320	Kiara	22	Cloud	82
321	Lakshya	19	DS	80
322	Pari	19	Cloud	82
323	Rudra	22	Cyber	80
324	Ananya	21	Cloud	81
325	Shaurya	22	DS	87

Roll No	Name	Age	Dept	Marks
326	Ishani	19	IoT	91
327	Atharv	20	Cyber	63
328	Myra	19	Cloud	88
330	Sneha	20	IoT	84
331	Parth	20	Cyber	62
332	Trisha	22	Cloud	98
333	Krish	22	DS	89
334	Vedika	21	IoT	90
335	Neel	22	Cyber	97
336	Jiya	22	Cyber	80
337	Samar	22	DS	67
339	Om	19	Cyber	57
340	Aarna	20	Cloud	96
341	Devansh	22	DS	88
342	Mira	22	IoT	78
343	Hriday	21	IoT	99
344	Tara	22	Cloud	84
345	Ronit	22	DS	82
346	Zainab	19	IoT	73
347	Ayush	21	Cyber	84
348	Fatima	19	Cloud	83
349	Nitin	21	DS	83

Marks greater than 70 AND age less than 22:

Code:

```
db.students.find({
  $and: [
    { marks: { $gt: 70 } },
    { age: { $lt: 22 } }
  ]
})
```

Output:

```
> db.students.find({
    $and: [
      { marks: { $gt: 70 } },
      { age: { $lt: 22 } }
    ]
  })
{
  _id: ObjectId('6acf2ff1dd73cf256d8f4d3e'),
  rollNo: 301,
  name: 'Aarohi',
  age: 20,
  department: 'DS',
  marks: 78
}
{
  _id: ObjectId('6acf2ff1dd73cf256d8f4d43'),
  rollNo: 306,
  name: 'Anushka',
  age: 19,
  department: 'IoT',
  marks: 94
}
{
  _id: ObjectId('6acf2ff1dd73cf256d8f4d45'),
  rollNo: 308,
  name: 'Mahi',
  age: 19,
  department: 'Cyber',
  marks: 83,
  status: 'Active'
}
```

Figure 15: Marks greater than 70 AND age less than 22 - 20 documents matched (first 3 shown)

All matching records (20):

Roll No	Name	Age	Dept	Marks
301	Aarohi	20	DS	78
306	Anushka	19	IoT	94
308	Mahi	19	Cyber	83
309	Ayaan	19	DS	98
313	Kian	21	DS	77
316	Riddhi	21	Cloud	73
319	Yuvraj	21	Cyber	78
321	Lakshya	19	DS	80
322	Pari	19	Cloud	82
324	Ananya	21	Cloud	81

Roll No	Name	Age	Dept	Marks
326	Ishani	19	IoT	91
328	Myra	19	Cloud	88
330	Sneha	20	IoT	84
334	Vedika	21	IoT	90
340	Aarna	20	Cloud	96
343	Hriday	21	IoT	99
346	Zainab	19	IoT	73
347	Ayush	21	Cyber	84
348	Fatima	19	Cloud	83
349	Nitin	21	DS	83

Department is Cloud OR IoT:

Code:

```
db.students.find({
  $or: [
    { department: "Cloud" },
    { department: "IoT" }
  ]
})
```

Output:

```
> db.students.find({
    $or: [
      { department: "Cloud" },
      { department: "IoT" }
    ]
  })
{
  _id: ObjectId('6acf2ff1dd73cf256d8f4d3f'),
  rollNo: 302,
  name: 'Vihaan',
  age: 19,
  department: 'IoT',
  marks: 69
}
{
  _id: ObjectId('6acf2ff1dd73cf256d8f4d41'),
  rollNo: 304,
  name: 'Tanvi',
  age: 22,
  department: 'Cloud',
  marks: 92
}
{
  _id: ObjectId('6acf2ff1dd73cf256d8f4d43'),
  rollNo: 306,
  name: 'Anushka',
  age: 19,
  department: 'IoT',
  marks: 94
}
```

Figure 16: Department is Cloud OR IoT - 22 documents matched (first 3 shown)

All matching records (22):

Roll No	Name	Age	Dept	Marks
302	Vihaan	19	IoT	69
304	Tanvi	22	Cloud	92
306	Anushka	19	IoT	94
310	Prisha	22	IoT	85
312	Navya	22	Cloud	71
314	Saanvi	22	IoT	87
316	Riddhi	21	Cloud	73
318	Aditi	21	IoT	56
320	Kiara	22	Cloud	82
322	Pari	19	Cloud	82
324	Ananya	21	Cloud	81

Roll No	Name	Age	Dept	Marks
326	Ishani	19	IoT	91
328	Myra	19	Cloud	88
330	Sneha	20	IoT	84
332	Trisha	22	Cloud	98
334	Vedika	21	IoT	90
340	Aarna	20	Cloud	96
342	Mira	22	IoT	78
343	Hriday	21	IoT	99
344	Tara	22	Cloud	84
346	Zainab	19	IoT	73
348	Fatima	19	Cloud	83

Marks NOT greater than 80:

Code:

```
db.students.find({ marks: { $not: { $gt: 80 } } })
```

Output:

```
> db.students.find({ marks: { $not: { $gt: 80 } } })
{
  _id: ObjectId('6acf2ff1dd73cf256d8f4d3e'),
  rollNo: 301,
  name: 'Aarohi',
  age: 20,
  department: 'DS',
  marks: 78
}
{
  _id: ObjectId('6acf2ff1dd73cf256d8f4d3f'),
  rollNo: 302,
  name: 'Vihaan',
  age: 19,
  department: 'IoT',
  marks: 69
}
{
  _id: ObjectId('6acf2ff1dd73cf256d8f4d42'),
  rollNo: 305,
  name: 'Kabir',
  age: 22,
  department: 'DS',
  marks: 70
}
```

Figure 17: Marks NOT greater than 80 - 20 documents matched (first 3 shown)

All matching records (20):

Roll No	Name	Age	Dept	Marks
301	Aarohi	20	DS	78
302	Vihaan	19	IoT	69
305	Kabir	22	DS	70
307	Reyansh	19	Cyber	62
311	Dhruv	21	Cyber	68
312	Navya	22	Cloud	71
313	Kian	21	DS	77
315	Arjun	21	DS	60
316	Riddhi	21	Cloud	73
318	Aditi	21	IoT	56

Roll No	Name	Age	Dept	Marks
319	Yuvraj	21	Cyber	78
321	Lakshya	19	DS	80
323	Rudra	22	Cyber	80
327	Atharv	20	Cyber	63
331	Parth	20	Cyber	62
336	Jiya	22	Cyber	80
337	Samar	22	DS	67
339	Om	19	Cyber	57
342	Mira	22	IoT	78
346	Zainab	19	IoT	73

8. MongoDB Operator Summary

Operator	Meaning	Example
$gt	Greater than	marks: { $gt: 80 }
$lt	Less than	marks: { $lt: 80 }
$eq	Equal to	marks: { $eq: 85 }
$gte	Greater than or equal to	marks: { $gte: 80 }
$ne	Not equal to	marks: { $ne: 85 }
$and	All conditions must be true	marks > 70 AND age < 22
$or	At least one condition must be true	Cloud OR IoT
$not	Negates a condition	NOT (marks > 80)

Total dataset: 50 students inserted (Roll No. 301-350); 47 remain after deleting Roll No. 350 and the two students with marks below 40.

Implementation Details

- The project is implemented using MongoDB and executed through the MongoDB Shell (mongosh).
- A single collection named "students" is used, with fields: rollNo, name, age, department and marks.
- insertOne() and insertMany() are used to populate the collection with 50 student records.
- updateOne() and updateMany() with the $set operator are used to modify existing fields and add new ones (e.g. status).
- deleteOne() and deleteMany() are used to remove individual and bulk records based on a filter condition.
- find() is combined with comparison operators ($gt, $lt, $eq, $gte, $ne) and logical operators ($and, $or, $not) to retrieve specific subsets of student data.
- Each operation's acknowledgement object (acknowledged, matchedCount, modifiedCount, deletedCount) is used to verify that it executed as intended.

Results

All CRUD operations executed successfully on the students collection. The insertOne and insertMany operations populated the collection with 50 student records. The updateOne operation successfully changed one student's marks (72 to 78), while updateMany correctly modified all 12 Cyber-department documents to add an "Active" status. The deleteOne operation removed roll number 350, and deleteMany removed the 2 records with marks below 40 (roll 317 (35 marks) and roll 338 (38 marks)), leaving 47 documents in the collection. The query operations correctly filtered students by marks, age and department using comparison and logical operators: 27 students scored above 80 while 20 did not, 17 students scored below 80 while 30 scored 80 or more, 2 students scored exactly 85, and 22 students belong to the Cloud or IoT departments. These counts are consistent with each other (27 + 20 = 47 and 17 + 30 = 47), demonstrating that MongoDB's query engine returns accurate, condition-based results.

Conclusion

This project highlights the practical use of MongoDB as a NoSQL database for managing structured academic data. By implementing insertion, update, deletion and a wide variety of query operations on a students collection, the project demonstrates a solid, hands-on understanding of MongoDB's Create, Read, Update and Delete (CRUD) capabilities as well as its comparison and logical query operators. The approach shows how a document database can be used to store and efficiently retrieve real-world tabular data such as student records.

Future Scope

- Use the MongoDB Aggregation Framework to compute department-wise averages, toppers and rankings.
- Add schema validation rules to enforce data types and required fields at the database level.
- Create indexes on frequently queried fields (e.g. marks, department) to improve performance.
- Build a front-end / web dashboard (e.g. using Node.js and Express) to interact with the database visually.
- Extend the collection with additional entities such as courses, attendance and results, and model relationships between them.

References

- MongoDB Official Documentation: https://www.mongodb.com/docs/
- MongoDB CRUD Operations: https://www.mongodb.com/docs/manual/crud/
- MongoDB Query Operators: https://www.mongodb.com/docs/manual/reference/operator/query/

- mongosh (MongoDB Shell) Documentation: https://www.mongodb.com/docs/mongodb-shell/

THANK YOU!